SEASUG Paper 80-2026

One Code, Two Modes: A Convention-Driven R Reporting System for Submission-Ready Clinical TLFs

Jaime Yan, George Wu, and Jessie Yan, Kardigan

ABSTRACT

In a SAS® shop, submission-ready formatting is a solved problem: ODS templates and macro libraries give every program the same page geometry, fonts, and titles. In R, the building blocks exist — r2rtf, rtables, ggplot2, the wider pharmaverse — but teams still assemble them by hand: Shiny apps drift from batch programs, conventions live in comments, and title metadata is maintained manually.

We present a reporting system in which the same R builder functions produce the same submission-ready RTF outputs in scheduled batch production and interactive Shiny preview — one code path, two modes. Four design elements carry the system: a convention engine encodes page geometry, fonts, and formatting rules as reusable code — the R analogue of a SAS ODS/macro convention stack; a dispatcher routes typed requests to specialized builders, shrinking standalone programs to thin wrappers that preserve the one-program-one-output audit trail; a title resolver reads the same metadata database SAS programs use, with three-level graceful degradation; and figure rendering runs in isolated child processes with bounded retries, containing a Windows-specific graphics crash that R cannot catch.

The system provides 16 TLF templates (6 tables, 8 figures, 2 listings), verified by execution against production study data on the validated server: 11 of 16 produce submission-formatted RTF unchanged; the five failures are data-shape adaptations, not code defects. A side-by-side SAS comparison protocol — numeric match at displayed precision, character-identical title blocks, documented exemptions — is defined, with full execution in progress. The architecture transfers to any organization building R-based clinical reporting alongside SAS.

1. INTRODUCTION

1.1 THE TLF CHALLENGE

A single clinical study easily requires a hundred or more Tables, Listings, and Figures (TLFs). Every one of them must obey the same formatting conventions — page geometry, fonts, title-block layout, footnote style — and every one must be traceable: which program produced it, from which data, with which titles, at which milestone. In a regulated environment these are not aesthetic preferences; they are audit requirements.

1.2 WHAT SAS SOLVES — AND WHAT R LEAVES TO EACH TEAM

In a mature SAS shop this problem is solved twice over. ODS destinations and style templates fix the page geometry and fonts; a macro library — an internal `%toc_map()`-style macro, an RTF template macro, an autocall path — gives every program the same title resolution and the same rendering. A new programmer inherits the conventions by inheriting the macros.

R has excellent building blocks: r2rtf renders submission-style RTF, rtables and gtsummary construct tables, ggplot2 builds figures, and the wider pharmaverse ecosystem supplies much more. What R does not supply is the convention stack around them. Each team assembles the pieces by hand, and three predictable failure modes follow. Exploratory Shiny apps drift away from the batch production programs they were meant to preview, because the two are separate codebases. Formatting conventions live in comments and slide decks rather than in code, so compliance depends on each programmer's memory. And title and footnote metadata — which in the SAS world comes from a single table of contents (TOC) — is re-typed by hand into each R program.

1.3 CURRENT R APPROACHES AND THEIR LIMITS

Published and open-source approaches each cover part of the problem: teal is interactive-first and does not produce locked, submission-ready RTF; tfrmt expresses a table's layout as metadata for pre-lock

review, but not the full page and figure conventions of a locked deliverable; rtables, gtsummary, and r2rtf are primitives, not a system — nothing in them decides that every figure title is Courier New 9 or that every page is landscape Letter with half-inch margins; targets and chevron orchestrate and scaffold but leave conventions to the user. Section 6.4 gives the full comparison.

1.4 DEPENDENCY DECLARATION

The system described in this paper runs on the centralized R infrastructure — automatic environment setup, version-locked package library, folder-convention path detection — presented in the companion SEASUG 2026 paper by the same authors; that infrastructure is assumed here and is not re-narrated.

1.5 GOAL: ONE CODE PATH, TWO MODES

Our goal was a reporting system in which a single R code path serves both scheduled batch production and interactive Shiny preview, with conventions encoded as code rather than comments, and with titles drawn from the same metadata source the SAS programs use. The rest of the paper describes the four design elements that carry the system — the dispatcher-builder architecture (Section 2), the convention engine (Section 3), the TOC-driven title resolver (Section 4), and crash-isolated figure rendering (Section 5) — followed by verification results (Section 6), lessons and limitations (Section 7), and future work (Section 8).

2. SYSTEM ARCHITECTURE

2.1 HIGH-LEVEL FLOW

Figure 1 shows the dual-mode architecture and the position of the convention engine and TOC resolver around the builders.

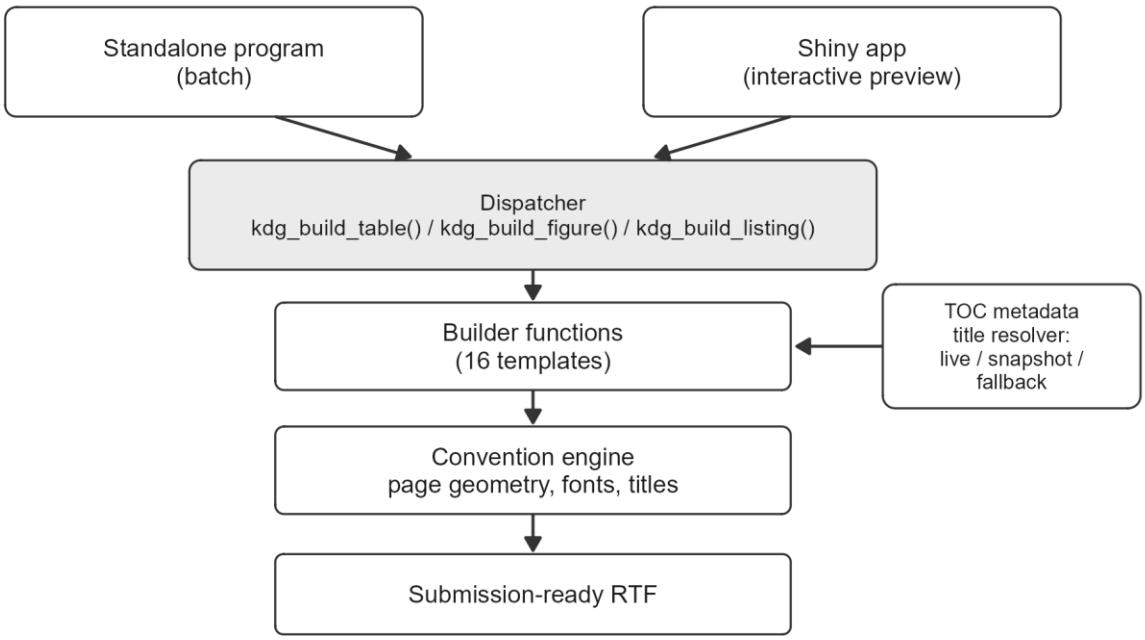


Figure 1. Dual-mode architecture. Analysis data flows into typed builder functions; builders return plain R objects; the convention engine (page constants, figure theme, RTF renderers, TOC resolver) turns those objects into submission-ready RTF. The batch path (Rscript on a scheduled job) and the interactive path (a Shiny reactive inside the reporting studio app) call the same builders and the same renderers.

Every builder obeys one contract: it takes already-read data frames and returns a plain result object; it never reads data, writes files, or touches the TOC. Analysis, rendering, and metadata are three separate seams, so the batch and interactive paths share all three without forking.

2.2 THE DISPATCHER-BUILDER PATTERN

We offer this pattern as a design rationale, not a novelty claim — typed dispatch over specialized functions is standard practice on both the SAS and R sides. What matters for the reader is the concrete shape it takes here. Each builder returns a fixed-shape result (a `kdg_table` object: a pre-formatted data frame, relative column widths, body justification, builder-derived footnotes, and a small provenance list), so every downstream renderer can be written once against that shape. A registry maps human-readable labels to type keys, and a dispatcher routes a typed request to the right builder:

```
kdg_build_table  <- function(type, data, opts = list())    # data: named
list of ADaM frames
kdg_build_figure <- function(type, df, ctx,  opts = list()) # ctx:
visit/treatment column mapping
```

Figure 2 traces one dispatcher call in each mode.

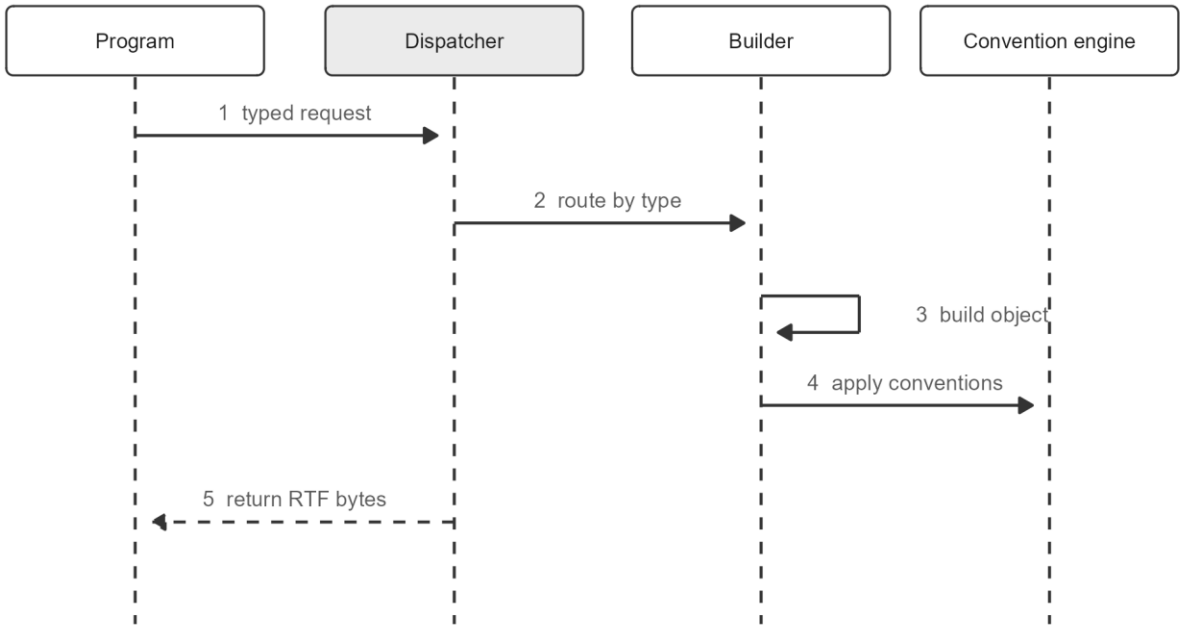


Figure 2. Dispatcher sequence, one call in each mode: the typed request, registry lookup, builder call, and convention application are identical whether the caller is a batch program or a Shiny reactive.

The figure engine currently registers 12 figure types (boxplot, mean with confidence interval in observed and change-from-baseline variants, spaghetti in both variants, LS-means in both variants, Kaplan–Meier, eDISH/Hy's Law, forest, waterfall, and shift); the table/listing engine registers 12 table and listing types. Not every registered type is yet productionized as a standalone template — Section 6.1 is explicit about the difference.

Exhibit 1 shows one complete builder, the demographics table builder used by template `t_demo.R`. The other builders follow the same shape and are available in the shared library by reference (`kdg_table_builders.R`, `kdg_fig_builders.R`).

```r
# Exhibit 1 — one builder in full (kdg_table_builders.R, de-identified
excerpt).
# kdg_tbl_demographics() stacks continuous blocks (n / Mean (SD) / Median
(Q1,Q3) /
# Min,Max, from kdg_desc_stats) and categorical blocks (n (%), from
kdg_freq_table),
# adds big-N column headers, and returns the standard kdg_table result.
kdg_tbl_demographics <- function(adsl, trt_var = "TRT01A",
                                 cont_vars = KDG_DEMO_CONT_DEFAULT,
                                 cat_vars  = KDG_DEMO_CAT_DEFAULT) {
  trt_var <- kdg_resolve_trt(adsl, trt_var)
  adsl <- kdg_filter_trt(adsl, trt_var)
  if (!nrow(adsl)) stop("No subjects with a non-blank '", trt_var, "'
value.", call. = FALSE)
  blocks <- list(); skipped <- character(0)
  for (v in names(cont_vars)) {
    if (!v %in% names(adsl)) { skipped <- c(skipped, v); next }
    blocks[[length(blocks) + 1L]] <-
      kdg_desc_stats(adsl, var = v, trt_var = trt_var, label =
cont_vars[[v]])
  }
  for (v in names(cat_vars)) {
    if (!v %in% names(adsl)) { skipped <- c(skipped, v); next }
    blocks[[length(blocks) + 1L]] <-
      kdg_freq_table(adsl, var = v, trt_var = trt_var, label =
cat_vars[[v]])
  }
  if (!length(blocks)) stop("No analysis variables present in adsl.", call.
= FALSE)
  tbl <- do.call(rbind, blocks)

  grp_cols <- setdiff(names(tbl), c("param", "label", "stat"))
  disp <- data.frame(
    Parameter = ifelse(!duplicated(tbl$param), tbl$label, ""),
    Statistic = tbl$stat,
    tbl[grp_cols], check.names = FALSE, stringsAsFactors = FALSE)

  bigN <- kdg_big_n(adsl, trt_var)
  for (g in grp_cols)
    if (g %in% names(bigN)) names(disp)[names(disp) == g] <- sprintf("%s
(%s)", g, bigN[[g]])

  kdg_table_result(disp,
    col_rel_width = c(2.2, 1.6, rep(1, ncol(disp) - 2)),
    text_justification = c("l", "l", rep("c", ncol(disp) - 2)),
    meta = list(trt_var = trt_var, skipped = skipped, groups = grp_cols))
}
```

Two details are worth noting for a SAS audience. First, the treatment variable is *resolved*, not assumed: `kdg_resolve_trt()` takes the requested variable if present and otherwise walks a candidate list (`TRT01A`, `TRTA`, `TRT01P`, `ARM`, `ACTARM`, `COHORT`, …), mirroring the auto-detect net the original standalone SAS programs carried. Second, builders fail loudly on empty or malformed input (`stop()` with a specific message) rather than emitting a silently wrong table.

2.3 THE THIN-WRAPPER PROGRAM PATTERN

Because the builders do the analysis and the renderers do the formatting, a standalone production program shrinks to a thin wrapper: read data, call the builder, resolve titles, render. Exhibit 2 is the

complete demographics template program (`t_demo.R`, de-identified: the study-specific header block, the configuration-file path, and study-specific parameter variants are removed or generalized; the logic is verbatim).

```
# Exhibit 2 — one thin-wrapper program in full (Table/t_demo.R, de-
identified).

#--- CLEAN WORKSPACE ---
rm(list = ls())

#--- ENVIRONMENT SETUP (auto-sources the shared R library, incl.
kdg_table_builders.R) ---
source("<shared-config>/rconfig.R")    # path generalized for publication
env <- setup()

#--- ANALYSIS PARAMETERS (edit per study) ---------------------------------
----
TRT_VAR   <- "TRT01A"
CONT_VARS <- list(AGE = "Age (years)")
CAT_VARS  <- list(SEX = "Sex", RACE = "Race", ETHNIC = "Ethnicity")

#--- READ + BUILD (shared builder — the same function the Shiny app calls)
-----
adsl <- env$read("adam", "adsl")
res  <- kdg_tbl_demographics(adsl, trt_var = TRT_VAR,
                             cont_vars = CONT_VARS, cat_vars = CAT_VARS)

#--- TOC + RENDER ---------------------------------------------------------
-----
toc <- toc_map_r(env, programname = env$pgmname, type = "Table", datain =
res$df)
toc$footnotes <- c(toc$footnotes, res$footnotes)
kdg_rtf_table(res$df, file = file.path(env$tflout, toc$output_file_name),
toc = toc,
              col_rel_width = res$col_rel_width, text_justification =
res$text_justification)
message("NOTE: wrote ", toc$output_file_name)
```

The audit-trail property SAS programmers rely on — one program, one output, a program header you can point an inspector at — survives intact. What disappears is the duplicated scaffolding: page setup, font choices, title assembly, and output-file naming all come from the shared library and the TOC, so two programs can no longer diverge on them. The output file name itself comes from the TOC (`toc$output_file_name`), which keeps the R deliverable aligned with the SAS naming convention for the same output.

2.4 DUAL-MODE EXECUTION AND THE SCOPE OF "IDENTICAL"

The interactive reporting studio calls the same builders and renderers through the same dispatcher: a Shiny reactive assembles the data list and options, calls `kdg_build_table()` or `kdg_build_figure()`, and renders through the same `kdg_rtf_*` functions the batch program uses. When we say the two modes produce *identical outputs*, we mean precisely this: **the same builder called with the same data produces the same RTF content in both modes.** The RTF bytes that carry the body, titles, footnotes, and embedded figure are generated by one code path. "Identical" does *not* extend to preview chrome — the Shiny UI framing, notification toasts, or download dialogs that surround the file in interactive mode — which is not part of the deliverable and is excluded from the comparison. Section 6.2 defines the verification protocol that backs this claim.

3. THE CONVENTION ENGINE

The convention engine is the R analogue of the SAS ODS/macro convention stack: a small library that makes every output compliant by construction. It has four layers.

3.1 PAGE GEOMETRY CONSTANTS

A single constants list, `KDG_PAGE`, is the source of truth for the physical page: landscape orientation, Letter size (11 × 8.5 inches), 0.5-inch margins on all sides (in r2rtf's six-element margin order), and the figure body rectangle (9 × 6 inches) that fits inside those margins. Renderers reference the constants rather than re-stating numbers, so a convention change — say, a different margin for a particular deliverable — is a one-line edit in one file, not a search-and-replace across every program.

3.2 FIGURE BODY THEME

`theme_kdg_submission()` is a ggplot2 theme encoding the figure-body conventions: Arial at base size 10 (the convention permits dropping to 9 when a panel cannot fit 10, and the theme enforces that 9–10 bound with a `stopifnot`), white panel background with a thin black border, major gridlines only, black axis and legend text, legend at the bottom, and — critically — **no in-body title**. The title is not the plot's job: it is set in the RTF text layer by r2rtf, in Courier New 9, from the TOC (Section 4). This split mirrors how the SAS figure macros separate the graph body from the title block, and it is what makes titles machine-auditable.

3.3 TABLE AND LISTING RENDERING

`kdg_rtf_table()` renders a pre-formatted data frame to RTF through r2rtf: page setup from `KDG_PAGE`, a running page header (organization name left, "Page x of y" right), the TOC title block, column headers and body with builder-supplied relative widths and justification, footnotes, and a source line — all in Courier New 9 (8 for the source line). r2rtf addresses fonts by integer index — not a stable public contract — so the renderer resolves the Courier New index at runtime by name (`.kdg_courier()`), hedging against a future r2rtf release renumbering its table. `kdg_rtf_listing()` is a thin specialization: the same renderer with left-justified body and repeating page headers. Tables and listings need no graphics device and run in-process; figures are different, and Section 5 explains why.

3.4 FIGURE RENDERING WITH CRASH ISOLATION

`kdg_rtf_figure()` renders a ggplot to a PNG (via ragg at 300 dpi, with a cairo fallback and an explicit warning if ragg is absent), embeds it in the RTF with r2rtf, and applies the same page header, title block, and footnotes as the table path. Because the graphics-device step can crash the R process outright on Windows, this renderer runs the render core in an isolated child process with bounded retries. That mechanism, its limits, and its honest failure policy are the subject of Section 5.

3.5 CONVENTION MAPPING: SAS ELEMENT TO R ELEMENT

Table 1 maps the convention stack a SAS programmer already knows onto its R counterpart in this system. The point of the table is not that the R side is exotic — it is that every element of the familiar stack has exactly one home in the R library.

Table 1. Convention-mapping: SAS convention-stack element → R element in this system.

Convention element	SAS implementation	R element (this system)
Page geometry (orientation, page size, margins)	ODS RTF options / style template	`KDG_PAGE` constants (landscape Letter 11×8.5 in, 0.5 in margins), consumed by all renderers
Page header: organization + "Page x of y"	`title1` with page fields (RTF control words)	`.kdg_apply_page_header()` → `r2rtf::rtf_page_header()`, Courier New 9

Convention element	SAS implementation	R element (this system)
Output banner (“Table 14.1.1”) + title lines	`title2`–`title10` set by `%toc_map()`	`.kdg_title_lines()` → `r2rtf::rtf_title()`, fed by `toc_map_r()`
Footnotes	`footnote1`–`footnoteN` from TOC	`r2rtf::rtf_footnote()` from `toc$footnotes`, Courier New 9
Source line	footnote-level source convention	`r2rtf::rtf_source()`, Courier New 8
Table/listing body font and size	PROC REPORT style overrides	`r2rtf::rtf_body()`/`rtf_colheader()`, Courier New 9; font index resolved at runtime by `.kdg_courier()`
Column widths and justification	PROC REPORT `define` statements	Builder-returned `col_rel_width` / `text_justification`, passed through to r2rtf
Figure body fonts (axes, legend, ticks)	GTL / PROC SGPLOT style	`theme_kdg_submission()`: Arial 10 (→9), white background, legend bottom, no in-body title
Figure assembly (image + text layer)	RTF figure template macro embedding a graph image	`kdg_rtf_figure()`: ggplot → ragg PNG (300 dpi) → `r2rtf::rtf_read_figure()` embed (crash-isolated, §5)
Title/footnote metadata source	`%toc_map()` reading the TOC metadata table	`toc_map_r()` reading the same SQL metadata database (§4)
Convention distribution to every program	SAS autocall library / `%include`	Shared R library auto-sourced at session setup (companion paper)

3.6 POSITIONING AGAINST TFRMT

The closest overlap in the published landscape is tfrmt, which expresses a table’s *layout* as metadata — grouping, labels, formats, footnotes — so that layout can be reviewed and locked before the data are final. That is genuinely adjacent to what we call the convention engine, and we want to state the delta honestly rather than pretend there is none. tfrmt addresses the layout of an individual table, primarily in service of pre-lock review. Our engine additionally encodes the *page* conventions (geometry, margins, page header), the *figure* conventions (body theme, resolution, embedding), and the *metadata plumbing* (TOC-driven titles shared with SAS), all aimed at locked submission output. The two are closer to complementary than competing: a tfrmt-style layout layer could sit underneath our renderers, and we flag that as a plausible integration rather than a rebuttal.

4. TOC-DRIVEN TITLE RESOLUTION

4.1 THE SAS PRECEDENT

In the SAS reporting suite this system coexists with, an organization-internal macro (`%toc_map()`) resolves each program’s titles, footnotes, output number, and output file name from a central table of contents. The TOC lives in a SQL metadata database shared with the SAS reporting suite — a table

(dbo.bps_toc, approximately 321 rows at the time of writing) keyed by program, study, and milestone, with columns toc_id, program, study, milestone, output_type, output_number, output_title, output_subtitle, output_footnotes, and output_file_name. Multi-line cells use a backslash as the line delimiter, and output numbers are zero-padded segment-wise so that 14.3.2 sorts before 14.3.10.

Our R resolver, toc_map_r(), is the R equivalent of that macro. We make no “first published” claim: the macro it mirrors is organization-internal, and we are simply **not aware of a published R equivalent** that reads the same source and reproduces the same title blocks.

4.2 THREE-MODE RESOLUTION

Figure 3 shows the three-mode resolution flow.

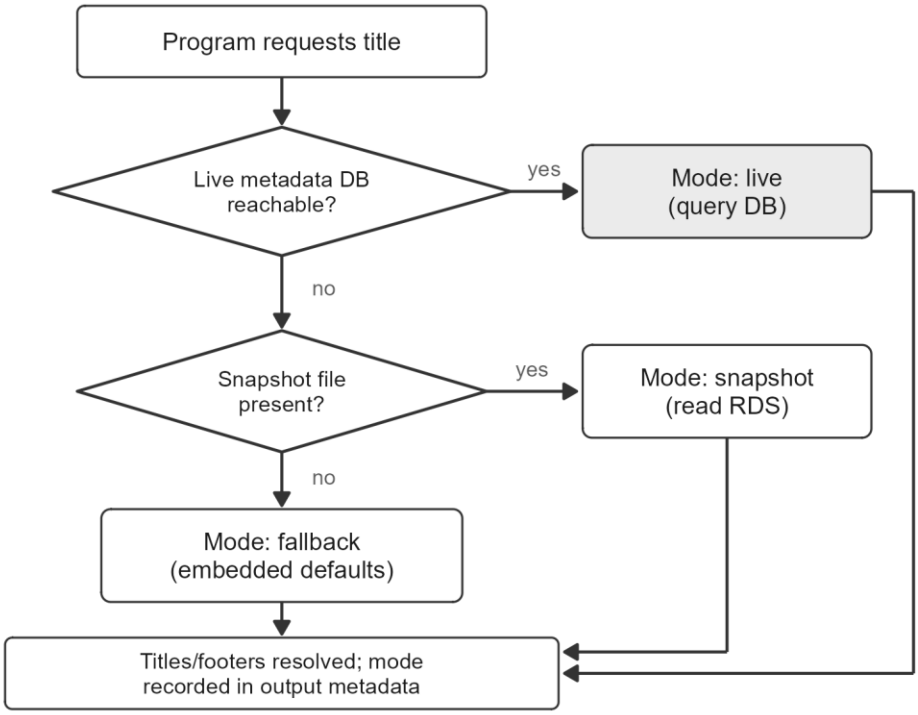


Figure 3. Three-mode TOC resolution flow: live database → local snapshot → embedded fallback, with the mode used recorded in each output’s metadata.

The resolver degrades gracefully through three modes, implemented in kdg_toc_utils.R:

Live. If the DBI and odbc packages are available and credentials are set (via environment variables — credentials are never hard-coded; the default account is read-only), the resolver opens a connection to the SQL metadata database and reads the full TOC table.

Snapshot. If live access fails or is unavailable, the resolver falls back to a local snapshot of the TOC (RDS or CSV), located via an environment variable or a conventional path in a shared network location.

Fallback. If neither source yields data, the program still runs: the resolver returns a TODO-marked placeholder banner (“Table x.y.z” plus a # TODO: <program> title from TOC line) and issues a warning. A dry run is never blocked by metadata access.

Within a retrieved TOC, the resolver matches the calling program by name inside the program/study/milestone scope inferred from the folder structure (the same hierarchy the companion paper’s infrastructure detects), falling back to an unscoped match if the scope is empty. It also preserves

the SAS macro's validation contract: the analysis data behind the output is saved to a validation dataset on each run.

4.3 TITLE FORMATTING: BYTE-FAITHFUL TO THE SAS MACROS

The formatting logic deliberately re-implements the SAS text handling, because parity is the goal, not approximation. Title, subtitle, and footnote cells are split on the backslash delimiter; the output number is zero-padded segment-wise (14.3.2 → sorts before 14.3.10); and the first title line is the type banner — "Table 14.1.1", "Figure 14.3.1", "Listing 16.1.1" — where the type defaults from the program's folder type (T → Table, F → Figure, L → Listing). The resolver returns a single list consumed directly by the renderers: the title vector (banner first), footnotes, study identifier and extraction date (rendered as leading title lines), the organization name for the page header, the source line, the output file name, and the resolution mode.

4.4 SAS–R PARITY FROM ONE SOURCE

The consequence of reading the same database is that SAS and R programs produce **character-identical title blocks** for the same output: same banner, same title lines, same footnotes, same file name — because there is literally one source string for each. This converts a class of QC finding ("title on the R version doesn't match the SAS version") from a recurring manual check into an impossibility, up to database availability. The comparison protocol in Section 6.2 verifies character-identity rather than assuming it.

4.5 SNAPSHOT GOVERNANCE

Three rules keep the snapshot mode honest. First, *live is authoritative*: whenever the database is reachable, the live read wins and the snapshot is not consulted. Second, *the mode is recorded*: the resolver returns the resolution mode (`live`, `snapshot`, or `fallback`) in its result, and the mode used for each output is recorded in that output's metadata — symmetric with the library-version metadata the companion paper records for reproducibility. Anyone reviewing an output can tell whether its titles came from the authoritative source or a point-in-time copy. Third, *refresh is event-driven*: snapshots are refreshed from the live database at defined points in the study lifecycle (e.g., before a dry run or an interim deliverable) rather than on a fixed clock. When a refresh is missed and the database has moved on, the divergence surfaces at QC as a title-block difference between snapshot-mode outputs and live-mode outputs — a visible discrepancy, not a silent one. The formal refresh cadence and divergence-tolerance policy are part of the validation framework being finalized alongside the comparison protocol (Section 6.2).

4.6 GRACEFUL DEGRADATION IN PRACTICE

The fallback mode exists for a specific operational reason: dry runs and early development happen before TOC rows exist, and sometimes off-network. A programmer can develop and execute a template end-to-end with no database access at all; the output carries an unmistakable TODO banner and the log carries a warning, so a placeholder title can never masquerade as a resolved one. The same mechanism covers the "row not yet entered" case when the database is reachable but the output has not been registered.

This behavior was verified on the production server on 2026-09-28: with the workspace's only TOC snapshot (54 rows) in place, titles resolved byte-exactly from it in snapshot mode; with the snapshot renamed away, the resolver degraded to the TODO banner, logged a warning, and the program continued to completion. The test also surfaced an open defect we record honestly: the resolver's default snapshot filename expects an RDS file while the deployed snapshot is CSV, so without the documented `TOC_SNAPSHOT` environment variable the snapshot mode does not engage (resolution degrades to fallback, with the warning). It is exactly the class of fault the three-mode design exists to make visible.

5. CRASH-ISOLATED FIGURE RENDERING

5.1 THE PROBLEM: AN INTERMITTENT NATIVE CRASH IN THE WINDOWS GRAPHICS STACK

The figure render path — ggplot built in memory, drawn to a PNG through ragg/systemfonts, read back and embedded into RTF by r2rtf — intermittently terminates the R process with a native crash (SIGSEGV, exit code 139) on our Windows development host. During development the trigger was correlated with nothing in our code: the evidence points at host state, plausibly font-cache I/O in systemfonts interacting with antivirus scanning or cloud-sync file locking during the render window. We describe this as a Windows-specific failure mode because our Linux deployment target has not exhibited it; we do not claim it is impossible elsewhere.

Two properties of the crash drive the design. It is *intermittent*: on the development host, the identical render path — same code, same data — went 6-for-6 clean and then, within minutes and with zero code change, crashed 0-for-N; isolated halves of the pipeline later went 8-for-8 and 10-for-10 clean. And it is *below R*: a segfault is not an error R can see.

5.2 WHY IT CANNOT BE CAUGHT IN-PROCESS

R's condition system (`tryCatch`, `withCallingHandlers`) handles errors and warnings raised at the R level. A native segmentation fault is an operating-system signal delivered to the process: there is no R condition to catch, no handler to invoke — the process simply dies. In an interactive Shiny session this kills the worker mid-download; in a batch job it kills the job. Any containment strategy therefore has to move the risky work out of the process whose survival matters.

5.3 THE PATTERN: A SHORT-LIVED CHILD PROCESS WITH BOUNDED RETRIES

We present this as a documented workaround, not a novel pattern: running untrusted or fragile work in a child process is precisely what callr is designed for, and pipeline tools such as targets use subprocess isolation for similar reasons. `kdg_rtf_figure()` ships a self-contained render core (arguments plus fully-qualified `pkg::` calls only, so callr can ship it to a fresh child session) to `callr::r()`, wraps the call in a retry loop, and returns the output path on the first success:

```
# Dispatcher-to-renderer seam (excerpt, kdg_rtf_utils.R) — the crash-
isolation boundary.
kdg_rtf_figure <- function(plot, file, toc = list(),
                           fig_width  = KDG_PAGE$fig_width,
                           fig_height = KDG_PAGE$fig_height,
                           res = 300, orientation = KDG_PAGE$orientation,
                           retries = 2L, timeout = 90) { ... }
```

A child crash surfaces to the parent as an ordinary (catchable) error object, which the loop counts and retries; a success returns immediately.

5.4 FAILURE POLICY — AS IMPLEMENTED, AND WHAT IS STILL FUTURE WORK

We state the implemented policy precisely, because earlier descriptions of this system overpromised here. As implemented in `kdg_rtf_utils.R`:

Bounded retries: `retries = 2L`, i.e., up to 3 total attempts.

Per-attempt timeout: 90 seconds, enforced by `callr::r(timeout = 90)`; a hung render counts as a failed attempt.

On exhaustion: the renderer raises an R error naming the number of attempts and the last child's error message. The batch program fails loudly; nothing silently succeeds.

No backoff between attempts; **no deterministic-crash detection** (a render that will crash on every retry is retried the full bound); **no alternate-renderer fallback in the batch path.**

A minimal valid fallback RTF on error exists only in the interactive Shiny path (the figure studio app), so an interactive user receives a well-formed placeholder document instead of a dead download button.

An honest caveat on containment: child-process isolation reliably converts *catchable* render errors and most crashes into retriable events, but it does not reliably contain every native crash on this Windows

host — font-touching operations outside the child can still fault the parent in the same hostile window. Escalation paths if Windows must be hardened further (a persistent `callr::r_session` pool, or a detached processx render with hard-kill and restart) are noted as options, not implemented.

Three items are explicitly **future work**, not current behavior: (i) backoff between retries, (ii) deterministic-crash detection (recognizing a repeat-offender render and short-circuiting), and (iii) a batch-path fallback renderer so an exhausted retry can still emit a reviewable placeholder RTF. See Section 8.

5.5 OBSERVED BEHAVIOR

What we can cite today are controlled measurements on the production host plus development-time observations, not yet live-workload telemetry. The development observations: the intermittent crash behavior described in 5.1 (6/6 clean flipping to 0-for-N within minutes; later 8/8 and 10/10 clean runs of isolated renders); and device-free unit render suites for the figure builders, all passing — forest 19/19, waterfall 13/13, shift 16/16, Kaplan–Meier 17/17, LS-means 12/12. These establish that the builders are correct and that the crash is host-level nondeterminism rather than a builder defect. The measurements use the attempt-level instrumentation built for exactly this purpose — the renderer accepts an optional log_file argument that appends one CSV row per attempt (timestamp, target, attempt number, status, error), added 2026-09-26. On the production Windows server itself (single enforced R version 4.5.1, ragg available): 200 consecutive isolated renders of a representative mean±SE figure (the f_meanci shape) succeeded 200/200, every one on the first attempt, at a mean of 1.9 s per isolated render including child-process startup (raw attempt log and summary retained by the authors, collected 2026-09-26). A companion development-host run (R 4.6.0) showed the same pattern at smaller scale (50/50, all attempt 1, 2.0 s; raw logs retained by the authors). Two honest readings: by the rule of three, 200 clean first-attempt renders bound the per-render crash probability for this render profile at 1.5% or below — but crashes cluster in bursts (5.1), so a burst can arrive after any clean streak, and with zero first-attempt failures observed, the retry path was never exercised: the retry success rate is not estimable from these runs. What the runs do pin down is the cost of isolation on the production host — about two seconds per figure, invisible against batch runtimes. Live-workload statistics — the production crash rate, the retry success rate, and the fallback invocation counts under real batch load — are not yet available; the attempt-level instrumentation described above is the mechanism that will collect them.

5.6 WHY THE BOUND IS THREE: A LABELED SIMULATION

The retry bound is a design choice, and we justify it with an explicit simulation rather than an appeal to experience. Let p be the per-attempt failure probability of one isolated render. Under the optimistic model — attempts fail independently — the probability that a render succeeds within k attempts is 1 − p^k. Under the pessimistic model — failures are perfectly correlated, i.e. a deterministic crash — no bound helps: success stays at 1 − p for any k. Table 2 shows both columns, with a Monte Carlo check (100,000 simulated render sequences per cell, seed fixed) reproducing the analytic independent-model values to sampling error; the simulation script and full results grid are retained by the authors (2026-09-26).

Table 2. Probability a render succeeds within *k* attempts, by per-attempt failure probability *p* and failure model.

p	*k* = 1 (indep.)	*k* = 2 (indep.)	*k* = 3 (indep.)	any *k* (correlated worst case)
0.01	0.990	0.9999	>0.9999	0.99
0.05	0.950	0.9975	0.9999	0.95
0.10	0.900	0.990	0.999	0.90
0.20	0.800	0.960	0.992	0.80
0.30	0.700	0.910	0.973	0.70
0.50	0.500	0.750	0.875	0.50

Two conclusions. First, moving from k = 1 to k = 3 captures nearly all of the independent-model gain — at p = 0.20, success rises from 0.80 to 0.992, while k = 5 would add only 0.008 — so a bound of three is the

knee of the curve, not an arbitrary choice. Second, our observed crash pattern (5.1: clean streaks flipping to burst failures within minutes) sits between the two columns, and the correlated column is flat in k: against burst failures, a larger bound buys nothing. That is exactly why deterministic-crash detection and a batch fallback renderer (5.4) are the correct next investment, rather than a bigger retry count. We stress the epistemic status of this table: it is a design argument over a parameter grid, not a claim about the production value of p; the production estimate of p is the live-workload measurement deferred to future work in Section 5.5.

6. RESULTS & COMPARISON

6.1 TEMPLATE INVENTORY — HONESTLY FRAMED

The system ships **16 template programs**: 6 tables, 8 figures, and 2 listings, enumerated with their builders in Appendix A. Two honest caveats. First, the engines underneath register more types than are productionized — the figure engine supports 12 figure types and the table/listing engine 12 types; the 16 templates are the subset that has been hardened into standalone programs with TOC wiring. Second, 16 templates do not cover a real study: with 100 or more outputs per study, these templates cover the common skeleton (demographics, disposition, adverse events, shift, time-to-event, efficacy summaries; the standard safety and efficacy figure set; the two standard listings), and study-specific outputs still require bespoke builders written against the same contracts. The claim is a working system plus a repeatable way to extend it, not full coverage.

Execution on production data (Table 3). All 16 templates were executed on the production server against a real study milestone's dry-run data on 2026-09-28. Eleven produced submission-formatted RTF unchanged (3–8 s per program, median 5 s). Five failed, and all five failures are **data-shape adaptations, not code or environment defects**: two reference a time-to-event ADaM dataset the milestone does not contain; three meet data shapes outside the templates' default statistical assumptions (no post-baseline visits to summarize; no estimable subgroup for the forest plot; LS-means not estimable). Two disclosures are owed. First, to respect the write-nothing-in-production rule of the evidence collection, the templates ran as sandbox copies with their data folders junction-linked read-only to the production milestone and outputs written to a sandbox directory; no production file was written, while data, package library chain, and TOC metadata were the production originals. Second, the failures are informative rather than embarrassing: they mark exactly where study-specific adaptation begins — which is the boundary this section claims.

Table 3. Execution of the 16 templates against production dry-run data, 2026-09-28.

Template	Type	Result	Time (s)	Failure category
`t_demo`	Table	PASS	6	—
`t_ae`	Table	PASS	4	—
`t_disposition`	Table	PASS	4	—
`t_eff`	Table	FAIL	4	no post-baseline visits
`t_shift`	Table	PASS	4	—
`t_tte`	Table	FAIL	4	ADaM dataset not present
`f_boxplot`	Figure	PASS	8	—
`f_edish`	Figure	PASS	7	—
`f_forest`	Figure	FAIL	4	no estimable subgroup
`f_km`	Figure	FAIL	3	ADaM dataset not present
`f_lsmeans`	Figure	FAIL	4	LS-means not estimable
`f_meanci`	Figure	PASS	6	—

Template	Type	Result	Time (s)	Failure category
f_spaghetti	Figure	PASS	7	—
f_waterfall	Figure	PASS	6	—
l_ae	Listing	PASS	4	—
l_lab	Listing	PASS	8	—

6.2 VERIFICATION PROTOCOL

The parity claims in this paper are only as good as their comparison protocol, so we define it here (it is shared with the companion paper's validation framework):

Numeric rule. Every numeric cell in the R output must match the SAS-produced equivalent **at displayed precision**, with edge cases handled explicitly: zero (including the zero-percentage convention, where a zero count prints as `0` rather than `0 (0.0%)`), negative zero (a `-0.0` on one side and `0.0` on the other is a *fail*, not a pass), and missing values (blank versus `NE` versus `.` conventions must agree cell-for-cell).

Title-block rule. The full title block — banner, title lines, footnotes, source line — must be **character-identical** between the SAS and R outputs, verified by field-level diff against the TOC-derived strings.

Documented exemptions. Exemptions are enumerated per template *before* comparison and recorded with the results — for example, the extraction-date line, which legitimately differs when the two runs happen on different days, and RTF control-word ordering differences that do not change rendered content. Anything not on the exemption list is a failure.

Per-template pass results under this protocol require a same-specification SAS reference set, and the honest status is:

A full-tree scan of the production server on 2026-09-28 (1,161 RTF files) found exactly one name-matched SAS output pair for the 16 templates — and its table specification differs from the R template's (a study-specific cohort layout versus the template's standard variable set), so the pair is not comparable and no exemptions were written: 0 of 1 candidate pairs are comparable. The protocol above stands, and the comparison harness exists and is tested (srv_compare_rtf.R: numeric tokens compared in reading order at displayed precision; non-numeric text compared character-identically after documented exemptions); executing the protocol across the 16 templates requires generating a same-specification SAS reference set, which is in preparation.

6.3 SAS PARITY EVIDENCE

What exists today: (i) the **structural argument** — title blocks are character-identical by construction, because both languages read one metadata source (Section 4.4), and the snapshot-mode test of Section 4.6 demonstrates byte-exact resolution from a real 54-row TOC snapshot on the production server; (ii) the **execution evidence** of Table 3; (iii) the **protocol and tested harness** of Section 6.2. What does not yet exist is the same-specification SAS reference set needed to run the protocol across the 16 templates — the only name-matched SAS output found on the server uses a study-specific specification and is not comparable. The parity claim is therefore a protocol plus a structural argument, with execution pending. We state this plainly because it is precisely the claim reviewers should push on, and we would rather lose the sentence than overclaim it.

6.4 COMPARISON WITH EXISTING FRAMEWORKS

teal (interactive-first). teal provides exploratory Shiny modules for clinical data review; it does not target locked, submission-ready RTF from the same code path, and batch production is out of scope. Our system is complementary in intent: interactive *preview of the production output*, not interactive exploration as the product.

tfrmt (layout-as-metadata). The closest overlap — see Section 3.6 for the honest delta. tfrmt encodes *table layout* as metadata for pre-lock review; our engine additionally encodes page geometry, fonts, figure conventions, and TOC-driven title resolution for locked submission output.

rtables / gtsummary (table construction). Both are table-construction layers; rtables pairs with r2rtf and carries a regulatory track record through the R Consortium Submissions Working Group pilots. They produce table bodies; they do not supply the page convention stack, the TOC integration, or the dual-mode execution that are the subject of this paper. Our table path renders plain pre-formatted data frames through r2rtf directly, a choice made for dependency minimalism rather than as a criticism of rtables.

chevron (pipeline scaffolding). chevron assembles standard TLF programs from parameterized building blocks — an adjacent answer to the duplication problem, oriented toward program generation rather than a shared runtime builder layer and a live/preview duality.

targets (orchestration). targets manages execution graphs, caching, and subprocess isolation; it answers "run the right things in the right order," not "what conventions does every output obey." A targets pipeline could orchestrate our thin wrappers without changing them.

R Consortium submission pilots (regulatory track record). The Submissions Working Group pilots established that r2rtf/rtables-based output can support a regulatory submission; our system builds on precisely that toolchain, and we treat the pilots as the evidence base for the renderer rather than re-litigating it.

Prior end-to-end accounts. The closest published system description is the Merck analysis-and-reporting SDLC (Wu et al. 2021): per-study R packages, `.Rprofile` environment setup, R Markdown TLF programs, and testthat-based validation, rendering RTF through r2rtf (Wang et al. 2020). That pipeline is batch-only and per-study packaged; the dual-mode execution, shared convention engine, and TOC-database integration described here are the delta.

The gap this system fills is the conjunction, not any single cell: one code path for batch and interactive, conventions as code, one metadata source shared with SAS, and crash-tolerant figure rendering on Windows.

7. LESSONS LEARNED AND LIMITATIONS

7.1 BUILDER FUNCTIONS, NOT MONOLITHIC SCRIPTS

The single highest-leverage decision was making builders pure functions of data. It is what makes dual-mode execution possible at all, what makes the device-free unit suites feasible, and what shrinks new outputs to thin wrappers. Monolithic programs that mix reading, analysis, and rendering cannot be shared between modes and cannot be tested without a graphics device.

7.2 CONVENTION AS CODE, NOT CONVENTION AS COMMENTS

Every convention that lived in a document or a comment has drifted in practice; every convention that lives in `KDG_PAGE`, `theme_kdg_submission()`, and the renderers cannot drift between programs, because programs never restate it. The font-index resolution hedge (`.kdg_courier()`) is the same lesson applied to a dependency's internals.

7.3 CRASH ISOLATION AS A PRODUCTION-GRADE PATTERN — WITH STATED LIMITS

Moving the graphics-device work into a child process converted an unrecoverable worker death into a countable, retriable event, and the retry bound keeps failures loud. The limits matter as much: isolation does not contain every native crash on Windows, host-level intermittency cannot be fixed in code, and render-stability verification belongs on the deployment host (Linux for us), with device-free tests carrying the correctness burden on the affected Windows host.

7.4 TOC AS THE SINGLE SOURCE OF TRUTH FOR METADATA

Reading titles from the same database the SAS programs use eliminated an entire QC category and aligned file naming for free. The discipline that makes it work is governance, not code: live-wins resolution, mode recording in output metadata, and event-driven snapshot refresh (Section 4.5).

7.5 LIMITATIONS

Scope. 16 productionized templates against the reality of 100+ outputs per study; bespoke outputs still require new builders (Section 6.1).

Windows-specific failure mode. The crash-isolation machinery exists because of a Windows graphics-stack fault; organizations rendering exclusively on Linux get a simpler system and should treat Section 5 as insurance, not necessity.

Proprietary SQL backend. Title resolution is tied to one organization's SQL metadata database and schema; the three-mode resolver generalizes (any DBI/ODBC source, any snapshot file), but the schema mapping is ours and would need re-pointing elsewhere.

ARD layer not yet integrated. Builders currently consume ADaM data frames directly rather than Analysis Results Data objects (cards/cardx); see Section 8 for the rationale and the path.

Infrastructure dependency. The zero-configuration experience assumes the companion paper's centralized environment layer (shared server, site file, configuration directory). The 16 templates and the shared library themselves are environment-agnostic: they deploy on any R installation with the required packages, losing only the transparent startup where that layer is absent.

Known open defect. The resolver's default snapshot filename expects `.rds` while the deployed snapshot is `.csv`; absent the `TOC_SNAPSHOT` variable, snapshot mode does not engage (found 2026-09-28; Section 4.6).

8. FUTURE WORK

Three extensions are planned, in order of expected value.

Metadata-driven program generation. Because every output is already a thin wrapper — parameters, builder call, TOC resolution, render — and the TOC already knows every output's number, type, and file name, the TOC can generate the program stubs: one wrapper per TOC row, pre-wired to the right builder, leaving only per-study parameters to fill in. This closes the loop from "TOC drives titles" to "TOC drives the program inventory."

ARD-layer integration (cards/cardx). For reviewers who ask why the ARD direction is not already integrated: the organization prioritized demonstrating SAS-parity verification of the core template set on direct-ADaM builders before inserting an intermediate results layer — sequencing, not rejection. The integration path is mechanical: builders gain an alternative input contract that accepts cards/cardx ARD objects in place of raw ADaM frames, with the renderers and TOC path unchanged.

Crash-policy hardening and portability. From Section 5.4: retry backoff, deterministic-crash detection, and a batch-path fallback renderer; optionally a persistent `callr::r_session` pool or detached processx render if Windows must be hardened further. Cross-organization portability follows from the seams the system already has — the convention engine is constants and themes, the resolver needs a schema mapping and a connection string, the builders are pure functions. What does not port is the TOC schema itself, which is why we describe the backend honestly as proprietary.

9. CONCLUSION

The broader conclusion is the one the title states. The SAS convention stack solved formatting consistency a generation ago by putting conventions in shared code and metadata in one table. Nothing about that solution is SAS-specific. With a dispatcher-builder architecture, a convention engine, a shared metadata resolver, and honest crash handling, one R code path can serve both the scheduled batch job and the interactive preview — and can hand a SAS audience outputs whose numbers and titles they already trust.

REFERENCES

Yan J, Wu G, Yan J. "Zero-Friction R for GxP: A Centralized Infrastructure Bridging SAS and R in Clinical Programming." Companion paper, SEASUG 2026 (submitted).

r2rtf package documentation. Merck & Co. / pharmaverse. https://merck.github.io/r2rtf/ — production-validated pin 1.3.0 (current CRAN release 1.3.1, accessed 2026-09-26).

rtables package documentation, version 0.6.16 (production pin = current release). Roche / pharmaverse. https://insightsengineering.github.io/rtables/ (accessed 2026-09-26).

tfrmt package documentation, version 0.4.0. GSK / pharmaverse. https://gsk-biostatistics.github.io/tfrmt/ (accessed 2026-09-26; not deployed in the production library).

cards (version 0.9.0) and cardx (version 0.3.4) package documentation. pharmaverse (ARD framework). https://insightsengineering.github.io/cards/ (accessed 2026-09-26; not yet deployed — see Section 7.5).

teal framework documentation. Roche / insightsengineering. https://insightsengineering.github.io/teal/ (accessed 2026-09-26).

R Consortium Submissions Working Group. Pilot submission reports (r2rtf/rtables-based regulatory submission track record). https://rconsortium.github.io/submissions-wg/ (accessed 2026-09-26).

Landau W. targets: Function Make-like Declarative Workflows — user manual, package version 1.12.0. https://books.ropensci.org/targets/ (accessed 2026-09-26; comparison reference, not deployed).

Csárdi G, Chang W. callr: Call R from R — package documentation. https://callr.r-lib.org/ — production-validated pin 3.7.6 (current CRAN release 3.8.0, accessed 2026-09-26).

Wickham H. ggplot2: Elegant Graphics for Data Analysis. Springer-Verlag (and package documentation, https://ggplot2.tidyverse.org/).

Therneau T. survival: Survival Analysis — package documentation. https://cran.r-project.org/package=survival.

Wu P, Palukuru UP, Luo Y, Nepal S, Zhang Y. 2021. “Analysis and Reporting in Regulated Clinical Trial Environment using R.” *Proceedings of the PharmaSUG 2021 Conference*, Paper AD-079. https://pharmasug.org/proceedings/2021/AD/PharmaSUG-2021-AD-079.pdf

Wang S, Ye S, Anderson KM, Zhang Y. 2020. “r2rtf — an R Package to Produce Rich Text Format (RTF) Tables and Figures.” *Proceedings of the PharmaSUG 2020 Conference*, Paper DV-198. https://pharmasug.org/proceedings/2020/DV/PharmaSUG-2020-DV-198.pdf

APPENDIX A: TEMPLATE CATALOG (16 PROGRAMS)

Tables (6):

Program	Output	Builder	Primary ADaM
`t_demo.R`	Demographics & baseline characteristics	`kdg_tbl_demographics()`	adsl
`t_disposition.R`	Subject disposition, n (%)	`kdg_tbl_disposition()`	adsl
`t_ae.R`	Treatment-emergent AEs by SOC/PT, n (%)	`kdg_tbl_ae()`	adae (+ adsl denominator)
`t_eff.R`	Efficacy: change from baseline by visit	`kdg_tbl_eff()`	BDS (e.g., adlb)
`t_shift.R`	Shift table: baseline × endpoint reference-range category	`kdg_tbl_shift()`	adlb
`t_tte.R`	Time-to-event: KM medians, Cox HR, log-rank p	`kdg_tbl_tte()`	adtte

Figures (8): *(the productionized subset of the 12 figure types the engine registers)*

Program	Output	Builder type
`f_meanci.R`	Mean (95% CI) by visit	`meanci`
`f_boxplot.R`	Box plot by visit	`boxplot`
`f_spaghetti.R`	Individual-subject lines with mean overlay	`spaghetti`
`f_lsmeans.R`	LS-means (baseline-adjusted) by visit	`lsmeans`
`f_km.R`	Kaplan–Meier survival curve	`km`
`f_forest.R`	Subgroup treatment-effect forest plot	`forest`
`f_waterfall.R`	Best % change per subject	`waterfall`
`f_edish.R`	Hy's Law / eDISH scatter	`edish`

Listings (2):

Program	Output	Builder
`l_ae.R`	Adverse event listing	`kdg_tbl_listing()` (AE column set)
`l_lab.R`	Laboratory result listing	`kdg_tbl_listing()` (lab column set)

CONTACT INFORMATION

Your comments and questions are valued and encouraged. Contact the authors at:

Jaime Yan
Kardigan
Email: jayan@kardigan.bio
George Wu
Kardigan
Email: gwu@kardigan.bio
Jessie Yan
Kardigan
Email: jyan@kardigan.bio